

Analisis de ingenieria inversa del APK sample.apk

Evaluacion completa de la aplicacion Android
com.example.lol: analisis estatico del manifiesto y del
snapshot Dart, inventario de endpoints y conexiones de red,
protecciones de evasion y analisis dinamico en emulador con
mitmproxy y Frida.

Veredicto: PUP de infraccion de copyright, riesgo medio

SHA-256: 4cefd5be...3cecf

Muestra: sample.apk (com.example.lol v1.0.0)

Indice

1. Resumen ejecutivo	2
2. Informacion del APK	2
3. Analisis del manifiesto	3
4. Analisis estatico de codigo	4
4.1 Arquitectura y ofuscacion	4
4.2 APIs sensibles y mecanismos internos	4
5. Analisis de red (estatico)	5
6. Protecciones identificadas	6
7. Analisis dinamico	6
7.1 Comportamiento observado	7
7.2 Trafico capturado	7
7.3 Comparacion estatico-dinamico y limitaciones	7
8. Indicadores de Compromiso (IoC)	8
9. Conclusiones y recomendaciones	8
9.1 Recomendaciones	9
Anexo A. Metodologia y herramientas	9

1. Resumen ejecutivo

Este informe documenta la auditoria de ingenieria inversa del archivo **sample.apk** (descarga original: https://modlyo.com/versiones/lol_200.apk). El objetivo del trabajo es determinar si la aplicacion es maliciosa, inventariar todas sus conexiones de red y evaluar las protecciones de evasion que implementa. La metodologia combina analisis estatico (apktool, jadx, androguard y extraccion de cadenas del snapshot Dart) con analisis dinamico en un emulador Android dentro de GitHub Actions, con interceptacion de trafico (mitmproxy) e instrumentacion (Frida).

Campo	Resultado
Veredicto	PUP: aplicacion de infraccion de copyright, sin conducta de malware clasico detectada
Tipo	Agregador de streaming pirata (scraper de enlaces + reproductor) para movil y Android TV
Nivel de riesgo	Medio
Riesgo de privacidad	Bajo: no recolecta datos personales mas alla del trafico de catalogo
Riesgo tecnico principal	Permiso REQUEST_INSTALL_PACKAGES + auto-actualizacion desde el backend propio sin verificacion de firma visible en el cliente

Tabla 1: sintesis del veredicto

La aplicacion, identificada como **com.example.lol** (etiqueta "lol"), es un cliente de streaming construido con Flutter que combina un catalogo legitimo (TMDB y OMDb) con enlaces de reproduccion obtenidos por scraping de al menos diez sitios pirata en espanol y una red de hosters de video. Toda la logica reside en el snapshot Dart AOT (libapp.so) y se apoya en un backend privado (modlyo.com) que actua como plano de control: decide los servidores disponibles, aloja subtítulos, valida la version instalada y puede forzar la actualizacion o desactivar fuentes de forma remota.

No se han detectado comportamientos tipicos de malware Android: sin acceso a SMS, contactos, ubicacion, camara ni microfono; sin SDK de analitica, publicidad ni telemetria; sin anti-analisis de ningun tipo; y sin exfiltracion de datos personales. Los riesgos se concentran en tres puntos: el canal de instalacion remota de actualizaciones, la infraccion sistematica de derechos de autor y la habilitacion de trafico en claro (usesCleartextTraffic=true) que permite interceptar la consulta de valoraciones a OMDb.

2. Informacion del APK

La muestra fue descargada del sitio del operador y su integridad fue verificada antes de cada fase del analisis. El archivo esta firmado con un certificado de depuracion creado dos semanas antes de la fecha de distribucion analizada y el nombre de paquete corresponde a una plantilla por defecto, lo que es coherente con compilaciones artesanales distribuidas fuera de tiendas oficiales, sin procedencia verificable.

Propiedad	Valor
Archivo	sample.apk (lol_200.apk); 21.853.058 bytes (20,8 MiB)
SHA-256	4cefd5bee99887fd1b0b06b545228695e4ed311949948d37ac922fdb09e3cecf

Propiedad	Valor
MD5	d59956c3990c7ae6613faaf89c194f36
Paquete	com.example.lol (etiqueta lol, versionName 1.0.0, versionCode 1001)
SDK	minSdk 24 (Android 7.0) / targetSdk 36 (Android 16)
Framework	Flutter (embedding v2); Dart AOT en libapp.so de 10,7 MB (solo armeabi-v7a)
Librerías nativas	libflutter.so (8,6 MB), libapp.so, libdartjni.so, libdatastore_shared_counter.so
Firma	APK Signature Scheme v2; certificado CN=Android Debug, O=Android, C=US, serie 0x1
Huella SHA-256 del certificado	44:ac:4c:9a:6b:22:95:96:31:3c:89:db:15:72:ab:8b:9b:fa:bb:35:a4:86:18:81:4a:0a:02:52:0e:db:82:07
Validez del certificado	2026-09-03 a 2056-08-26

Tabla 2: identificación del archivo analizado

La marca del operador aparece dentro de la propia aplicación junto a sus canales de difusión: el sitio modllyo.com (que además ejerce de backend) y las cuentas t.me/lol_oficialapp, instagram.com/lol_oficialapp y tiktok.com/@lol_oficialapp. El APK declara compatibilidad con Android TV (feature leanback y banner), lo que amplía la superficie de instalación típica de estas herramientas a televisiones y reproductores multimedia.

3. Analisis del manifiesto

El AndroidManifest decodificado (static/apk_extracted/AndroidManifest.xml) presenta una superficie de permisos contenida, con dos excepciones relevantes: el permiso de instalación de paquetes, que sostiene el mecanismo de auto-actualización, y la habilitación global de tráfico en claro. No hay servicios en segundo plano, receptores propios ni proveedores exportados más allá de los estándares de androidx.

Permiso	Riesgo	Uso previsto
INTERNET	Bajo	Streaming y APIs de catálogo
ACCESS_NETWORK_STATE	Bajo	Detección de conectividad
REQUEST_INSTALL_PACKAGES	Alto	Instalación de /lol_update.apk descargado del backend propio
WRITE_EXTERNAL_STORAGE (maxSdk 28)	Medio	Descargas en Android 9 o anterior
READ_EXTERNAL_STORAGE (maxSdk 32)	Bajo	Lectura de archivos descargados
READ_MEDIA_IMAGES/VIDEO/AUDIO	Bajo	Acceso a media en Android 13 o superior
WAKE_LOCK	Bajo	Pantalla encendida durante la reproducción
DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION	Informativo	Permiso de firma interno de androidx

Tabla 3: permisos solicitados y su evaluación

Los componentes exportados se reducen a MainActivity (lanzadores normal y leanback) y al receptor estandar ProfileInstallReceiver, protegido por el permiso privilegiado DUMP. Tres proveedores completan el manifiesto: dos FileProvider (uno con root-path="." declarado por el plugin open_file, alcance innecesariamente amplio) y el InitializationProvider de androidx. La configuracion de seguridad destaca por su ausencia: no hay networkSecurityConfig, allowBackup queda en su valor por defecto (true) y usesCleartextTraffic esta habilitado expresamente. Las queries declaran ACTION_PROCESS_TEXT (busqueda desde texto seleccionado) y ACTION_VIEW de APKs, coherente con el instalador.

4. Analisis estatico de codigo

4.1 Arquitectura y ofuscacion

El envoltorio Java/Kotlin (classes.dex, 2,2 MB, 1.530 clases) esta ofuscado con R8 en modo completo: paquetes y clases renombrados a una o dos letras (a.a, a0.o, f4.e) y mapa de nombres incrustado en los comentarios. Contiene unicamente el runtime de Flutter, los plugins y androidx/media3 (ExoPlayer con HLS y DASH). La decompilacion con jadx 1.5.1 solo presenta 13 errores menores, lo que descarta tecnicas anti-decompilacion. La totalidad de la logica de negocio reside en Dart compilado AOT dentro de libapp.so: no hay smali de la logica y el analisis se apoya en las 22.328 cadenas del snapshot. Las cadenas no estan cifradas: URLs, rutas de paginas Dart (package:lol/fuentes/apis/...), mensajes de interfaz y claves de configuracion aparecen en texto plano. No se detectan empaquetadores comerciales.

4.2 APIs sensibles y mecanismos internos

Categoria	Resultado del analisis
Red en capa Java	Solo stack de sistema; sin OkHttp ni Retrofit. La red real viaja por Dart HttpClient
Criptografia propia	No hay Cipher/SecretKeySpec; los hex de 32 caracteres son claves de API (TMDB) y constantes del runtime
Carga dinamica de codigo	Sin DexClassLoader, PathClassLoader ni InMemoryDexClassLoader
Reflexion	Limitada a androidx/window (reflectionguard), patron estandar
Ejecucion de comandos	Sin Runtime.exec ni ProcessBuilder
Datos personales	Ningun acceso a contactos, SMS, ubicacion, camara ni microfono
WebViews	Inyeccion de JavaScript con puente MediaDetector: vigila URLs .m3u8/.mp4/.ts/.m4s y las devuelve a Dart

Tabla 4: revision de APIs sensibles

El mecanismo de adquisicion de enlaces queda asi demostrado: la aplicacion carga paginas de los sitios fuente en WebViews invisibles, inyecta JavaScript que vigila las peticiones de medios y recibe las URLs por el puente MediaDetector. El flujo de actualizacion se apoya en cadenas propias (Actualiza para continuar, Descargar actualizacion, Error descarga APK) y en el archivo /lol_update.apk; la cadena CT_xor aparece junto a ese nombre y, por contexto, es un identificador de servidor, no un desempaquetador. Se identificaron cinco candidatos a clave de API de 32 caracteres hexadecimales, al menos uno de ellos correspondiente a la

clave v3 de TMDB.

5. Analisis de red (estatico)

El inventario estatico, extraido del snapshot Dart, del DEX y de los recursos, contiene 38 dominios y 130 URLs literales; no hay IPs operativas, ni WebSocket, ni MQTT, ni tunel DNS. La estructura de la comunicacion es clara: un plano de control propio (modlyo.com), APIs publicas de metadatos, diez sitios de streaming pirata raspados y una red de hosters que entregan el video al reproductor.

Endpoint	Metodo	Proposito	Riesgo
modlyo.com/apitv/version_api.php?version_code=100 y =200	GET	Validacion de version y entrega de /lol_update.apk	Medio-alto
modlyo.com/apitv/desactivar_servidor.php	GET/POST	Interruptor remoto de servidores y fuentes	Medio
modlyo.com/appapi/api_servidores.php?idcontenido=	GET	Servidores de reproduccion por titulo	Bajo-medio
www.modlyo.com/api/servidores.php	GET	Catalogo global de servidores	Bajo-medio
www.modlyo.com/apitv/apicuevana.php?type={movie,capitulo}&id;=	GET	Proxy de contenido tipo Cuevana	Bajo-medio
modlyo.com/subtitulo/contenido/	GET	Subtitulos alojados por el operador	Bajo

Tabla 5: plano de control del operador (modlyo.com)

Endpoint	Proposito	Riesgo
api.themoviedb.org/3/...?api_key= e image.tmbd.org	Catalogo y imagenes de TMDB	Bajo
http://www.omdbapi.com/?i=	Valoraciones; viaja en HTTP en claro	Medio
api.introdb.app	Intros de series para salto automatico	Bajo
opensubtitles-v3.strem.io/subtitles/{movie,series}/	Subtitulos (API v3 de OpenSubtitles)	Bajo

Tabla 6: APIs publicas de metadatos

Sitios raspados (fuentes)	Hosters y CDNs de reproduccion
www.pelisplushd.la; www.poseidonhd2.co; cinehax.com; wv3.cuevana3.eu; www.cinecalidad.am; serieskao.top; hackstore.mx; xupalace.org; lamovie.cc; net27.cc	embedwish.com; streamwish.to; strwish.com; awish.pro; wishfast.top; hanerix.com; hglink.to; unlimplay.com; dr0pstream.com; goodstream.one; embed69.org; tioplus.app; buzzheavier.com; pixeldrain.com

Tabla 7: sitios objetivo de scraping y hosters de video

Todo el trafico de control y catalogo viaja por HTTPS salvo la consulta a OMDb, que se realiza por HTTP en claro amparada en usesCleartextTraffic=true. No se identifican endpoints de exfiltracion de datos personales, servicios de paste, bots de Telegram para comando y control, ni farmacias de credenciales. La unicidad del trafico es funcional: catalogo, enlaces, subtitulos, trailers de YouTube y comprobacion de version.

6. Protecciones identificadas

La aplicacion no implementa ninguna capa deliberada de proteccion ni de evasion. La unica ofuscacion presente es la estandar de R8 en el envoltorio Java, y la opacidad de la logica proviene del modelo Dart AOT de Flutter, no de una tecnica defensiva. La tabla siguiente resume el inventario completo y la evidencia asociada, revisada sobre el DEX descompilado, el snapshot Dart y las librerias nativas.

Proteccion	Estado	Evidencia
Ofuscacion Java	Presente (R8 estandar)	Nombres a.a, f4.e; mapa r8 en las clases
Ofuscacion de logica	N/A (Dart AOT)	libapp.so sin simbolos Java
Cifrado de cadenas	Ausente	URLs y claves en texto plano en el snapshot
Empaquetadores	Ausentes	DEX normal, sin shells comerciales
Anti-debugging	Ausente	Sin isDebuggerConnected, TracerPid ni ptrace
Deteccion de root	Ausente	Sin su, magisk, busybox ni RootBeer
Deteccion de emulador	Ausente	Sin qemu, goldfish ni sdk_gphone
Anti-Frida / anti-Xposed	Ausente	Sin cadenas ni hooks defensivos
SSL pinning	Ausente	Sin CertificatePinner, TrustManager propio ni network_security_config
Anti-tampering	Ausente	Sin verificacion de firma propia en ejecucion
Carga dinamica de codigo	Ausente	Sin DEX loaders; la actualizacion instala un APK externo

Tabla 8: inventario de protecciones y evasion

Consecuencia practica para el analista: al no existir pinning, basta instalar la CA de mitmproxy en el almacen del sistema del emulador (procedimiento documentado en dynamic/README.md) para descifrar el trafico; no se requiere degradar TLS con Frida. El control real de la aplicacion esta del lado del servidor: la version minima forzada, el interruptor de servidores y la distribucion de actualizaciones permiten al operador gestionar el parque de instalaciones sin codigo local protegido. Este diseno traslada el riesgo a la cadena de confianza de la distribucion: el cliente no valida la firma del APK actualizado, por lo que un compromiso del backend se convertiria en ejecucion arbitraria en cada dispositivo que acepte el dialogo de actualizacion.

7. Analisis dinamico

El analisis dinamico se automatizo en un workflow de GitHub Actions (.github/workflows/dynamic-analysis.yml) sobre un emulador Android 11 (API 30, x86_64 con traduccion ARM ndk_translation 0.2.2), arranque headless con KVM, mitmproxy como proxy de red del emulador con su CA instalada en el almacen del sistema mediante bind mount, y frida-server 16.4.10 ejecutandose como root. Los hooks instrumentan java.net.URL, HttpURLConnection, Cipher, DexClassLoader, Runtime.exec, System.loadLibrary, SSLContext.init y getPackageInfo. El procedimiento de reproduccion manual esta documentado en dynamic/README.md.

7.1 Comportamiento observado

La aplicación se lanza correctamente mediante spawn de Frida. MainActivity completa onCreate, el motor Flutter carga libflutter.so y renderiza con Impeller sobre OpenGL ES bajo traducción ARM. Los hooks Java registran exactamente tres eventos iniciales: la carga de libflutter, la carga de libdartjni y una consulta getPackageInfo con flags=0. El registro automático de plugins aborta a continuación con UnsatisfiedLinkError: dlopen failed: library "libdartjni.so" not found (plugin dart_lang/jni), y la aplicación permanece en la pantalla de inicio sin completar la inicialización ni emitir una sola conexión de red, incluso con interacción automática (monkey, 400 eventos).

No se observan llamadas a DexClassLoader, Runtime.exec ni Cipher, ni muertes del proceso, ni reacciones anti-análisis de ningún tipo: root, Frida, la CA de mitmproxy y el proxy de red del emulador son aceptados sin resistencia. Este comportamiento confirma dinámicamente el inventario estático de protecciones.

7.2 Tráfico capturado

La captura completa (dynamic/captures/traffic.mitm.gz, 17,3 MB comprimidos; resumen legible en dynamic/captures/flows_log.txt) contiene 279 flujos descifrados y 45 intentos TLS fallidos, todos originados por componentes de sistema del emulador (Google Play Services, checkin y C2DM, YouTube, Firebase, verificación de conectividad y publicidad de sistema). Los códigos de respuesta dominantes son 200 (79), 204 (47), 400 (16) y 301/302 (12). Los fallos TLS corresponden a dominios de Google con anclaje interno (www.google.com, youtubei.googleapis.com, firebaseinstallations.googleapis.com, mtalk.google.com), que rechazan la CA de mitmproxy de forma esperable. Ningún flujo procede de la aplicación auditada.

7.3 Comparación estático-dinámico y limitaciones

Hallazgo estático	Resultado dinámico
Sin anti-debugging, anti-root ni anti-Frida	Confirmado: instrumentación completa sin resistencia
Sin SSL pinning en la app	No refutable (la app no emitió TLS); pinning observado solo en apps de sistema de Google
Sin SDK de telemetría propio	Confirmado: cero flujos originados por la app
Sin carga dinámica, sin exec, sin criptografía propia	Confirmado: hooks silenciosos en esas APIs
Backend modlly.com (version_api, servidores, interruptor)	No observable: la app no alcanza la fase de red en el entorno

Tabla 9: contraste entre análisis estático y dinámico

Limitaciones documentadas. Primera: el APK solo distribuye librerías armeabi-v7a y la traducción ARM del emulador x86_64 rompe la carga de libdartjni.so, dejando la inicialización incompleta y bloqueando la fase de red. Segunda: el subsistema de red virtual del emulador (netsim) se detiene unos 2,5 minutos tras cada arranque, reduciendo la ventana útil de observación. Tercera: los servicios de Google no son inspeccionables por pinning propio. Estas limitaciones no alteran el veredicto: las conductas relevantes están establecidas estáticamente sobre el snapshot Dart y el manifiesto, y la fase dinámica confirma la ausencia total de anti-análisis y de comportamiento oculto en todo lo observable.

8. Indicadores de Compromiso (IoC)

Los indicadores siguientes permiten detectar la muestra, su canal de distribucion y la infraestructura de control. Se recomienda bloquear el paquete y los dominios de control en controles corporativos, y usar la regla YARA de reports/evaluacion.md para detencion en repositorios de muestras.

```
SHA-256: 4cefd5bee99887fd1b0b06b545228695e4ed311949948d37ac922fdb09e3cecf
MD5: d59956c3990c7ae6613faaf89c194f36
Paquete: com.example.lol (etiqueta lol, versionCode 1001)
Cert: CN=Android Debug, O=Android, C=US, serie 0x1
Cert SHA-256: 44:ac:4c:9a:6b:22:95:96:31:3c:89:db:15:72:ab:8b:9b:fa:bb:35:
a4:86:18:81:4a:0a:02:52:0e:db:82:07
```

Tabla 10: identificadores del archivo y de su firma

```
modlyo.com / www.modlyo.com (backend de control)
https://modlyo.com/apitv/version_api.php?version_code={100,200}
https://modlyo.com/apitv/desactivar_servidor.php
https://modlyo.com/appapi/api_servidores.php?idcontenido=
https://www.modlyo.com/api/servidores.php
https://www.modlyo.com/apitv/apicuevana.php?type={movie,capitulo}&id;=
https://modlyo.com/subtitulo/contenido/
/lol_update.apk (artefacto de auto-actualizacion)
```

Tabla 11: infraestructura de control y distribucion

```
Sitios objetivo: www.pelisplushd.la www.poseidonhd2.co cinehax.com
wv3.cuevana3.eu www.cinecalidad.am serieskao.top hackstore.mx
xupalace.org lamovie.cc net27.cc
Hosters: embedwish.com streamwish.to strwish.com awish.pro
wishfast.top hanerix.com hglink.to unlimplay.com dr0pstream.com
goodstream.one embed69.org tioplus.app buzzheavier.com pixeldrain.com
Operador: t.me/lol_oficialapp instagram.com/lol_oficialapp
tiktok.com/@lol_oficialapp
```

Tabla 12: dominios objetivo, hosters y presencia del operador

9. Conclusiones y recomendaciones

La aplicacion "lol" no encaja en ninguna familia de malware Android: no roba datos, no espia, no cifra archivos, no instala nada de forma silenciosa por si misma y no evade el analisis. Es una herramienta de consumo de contenido pirata, sustentada en scraping y en un backend propio, con utilidades de gestion remota (version minima, servidores activos) propias de un servicio operado por un grupo concreto. La clasificacion correcta es PUP con infraccion de copyright y riesgo tecnico medio.

El riesgo relevante no esta en el código ejecutado hoy, sino en la cadena de confianza de la distribución: el operador puede empujar un APK cualquiera a la base instalada mediante la actualización forzada, y el cliente no valida la firma del paquete entrante. Un compromiso de modlyo.com, o de la red que conecta al cliente si el tráfico cayera a HTTP, habilitaría la ejecución arbitraria en cada dispositivo que acepte el diálogo de actualización. Además, el interruptor remoto de servidores demuestra capacidad de alterar el comportamiento de las instalaciones desplegadas a voluntad del operador.

9.1 Recomendaciones

Publico	Accion recomendada
Usuarios	No instalar la aplicación; si esta presente, desinstalarla y revisar aplicaciones instaladas de origen desconocido
Organizaciones y MDM	Bloquear el paquete com.example.lol, el dominio modlyo.com y los hosters listados en los IoC; alertar sobre APKs firmados con certificados Android Debug
Titulares de derechos	Notificar a los sitios fuente y al hosting de modlyo.com según sus procedimientos de abuso
Investigadores	Mantener la muestra aislada; el procedimiento de dynamic/README.md permite replicar el análisis completo
Ciclo de vida del análisis	Revocar y rotar credenciales utilizadas durante la auditoría una vez concluido el trabajo

Tabla 13: recomendaciones por público objetivo

Anexo A. Metodología y herramientas

Herramienta	Version	Uso
apktool	2.9.3	Decodificación de recursos y manifiesto
jadx	1.5.1	Descompilación del DEX (1.530 clases)
androguard	4.1.4	Metadatos, permisos y certificado de firma v2
strings/grep	sistema	Extracción de cadenas de libapp.so y del DEX
Scripts propios	scripts/	extract_strings.py y static_info.py (automatización)
mitmproxy	CI	Intercepción de tráfico HTTPS
Frida	16.4.10 (CI)	Instrumentación en tiempo de ejecución
Android emulator	API 30 (CI)	Ejecución controlada headless en GitHub Actions

Tabla 14: herramientas empleadas en cada fase

Limitaciones metodológicas: la lógica Dart no se descompila a fuente legible (snapshot AOT), por lo que el análisis de esa capa se basa en cadenas, comportamiento y tráfico; el análisis dinámico depende de la disponibilidad del backend del operador y de la estabilidad del entorno emulado (ver sección 7.3); y solo se analizó la variante armeabi-v7a incluida en el APK. Toda la evidencia cruda acompaña a este informe en los directorios static/ y dynamic/ del repositorio.